

ECOM EXPERTS

Ecom AdBlocker

Manual de usuario e instalacion

ecom_adblocker 1.3.1 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

Permisos que pide, y por qué

4. Configuración

El interruptor de arriba

Los dos contadores

Permitir en este sitio

Permitir anuncios en este canal

Las ocho listas

5. Cómo se usa, día a día

Navegación corriente

Cuando una web se rompe

Vídeo

Apoyar a un creador de YouTube

6. Qué ve el usuario en la web

7. Preguntas frecuentes y problemas

8. Seguridad

Lo que se ha arreglado

Lo que se ha revisado y estaba bien

Lo que queda pendiente

9. Ficha técnica

Ficheros

Ajustes guardados (chrome.storage.local)

Guiones de contenido y dónde entran

Reglas de red

Mensajes internos

Sin registros ni cron

Extensión de navegador (Chrome, Edge y Brave) para bloquear publicidad y rastreadores. Versión 1.3.1.

1. Qué hace y para quién

Ecom AdBlocker quita la publicidad de las páginas web antes de que llegue a cargarse. No la tapa después: corta la petición de red, de modo que el anuncio ni siquiera se descarga. El resultado es que las páginas pesan menos, cargan antes y consumen menos datos, además de verse limpias.

Junto con los anuncios bloquea los **rastreadores**, que son los pequeños ficheros que las webs cargan para seguirte de un sitio a otro y construir un perfil de lo que te interesa. Esa parte la cubre la lista EasyPrivacy, y se puede apagar por separado si en algún momento estorba.

La segunda capa se ocupa de lo que el bloqueo de red no puede quitar: los huecos vacíos que deja el anuncio, los banners que la propia web dibuja con su código, y los «wallpapers» o pieles publicitarias que decoran el fondo de la página. Esos se ocultan con reglas de estilo.

La tercera capa son los anuncios de vídeo de las plataformas de streaming. En YouTube, Disney+, Amazon Prime Vídeo y HBO Max los anuncios viajan por el mismo canal que la película o el vídeo, así que ningún bloqueo de red puede distinguirlos. Para cada una de esas cuatro plataformas hay un guion específico que detecta el anuncio y lo salta, lo silencia o lo tapa, según lo que se pueda hacer sin estropear la reproducción.

Está pensado para cualquier persona que navegue a diario y quiera una instalación que funcione sola, sin configurar nada, pero que permita hacer excepciones: dejar pasar la publicidad en una web concreta que te importe, o en los canales de YouTube de los creadores a los que quieras apoyar.

2. Compatibilidad

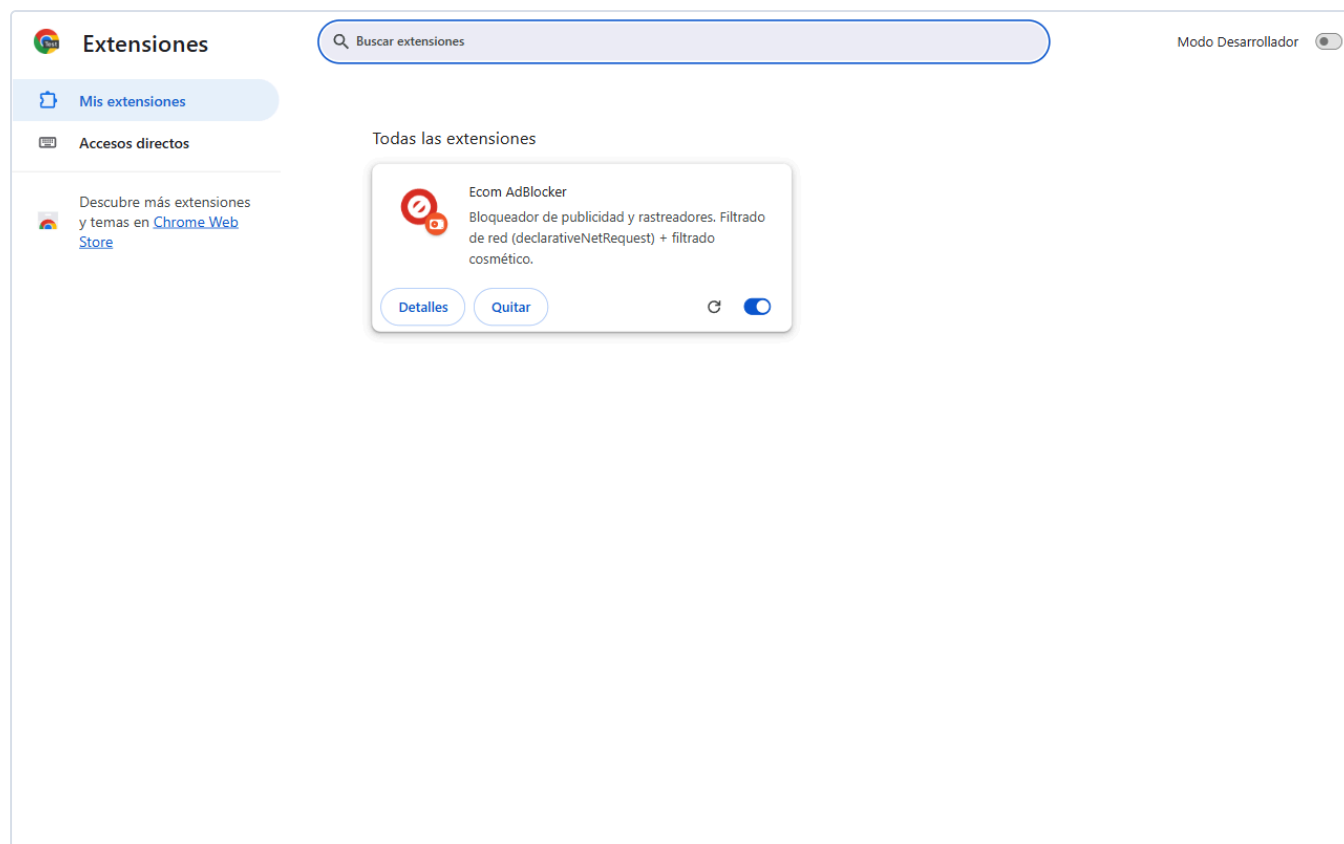
Navegadores	Chrome, Microsoft Edge, Brave, Opera y cualquier otro basado en Chromium
Versión mínima	Chrome 104 (declarada en el <code>manifest.json</code>)
Formato	Manifest V3
Firefox	No compatible. Firefox no implementa <code>declarativeNetRequest</code> igual y no tiene la API del contador
Dependencias	Ninguna. No usa librerías externas ni descarga nada
Idioma	Castellano

Sobre Firefox: aunque llegara a cargarse, la API `declarativeNetRequest.getMatchedRules()` que alimenta el contador de bloqueos no existe allí, de modo que el contador se quedaría siempre a cero.

3. Instalación

La extensión se entrega como carpeta, para cargarla en modo desarrollador:

1. Abre `chrome://extensions` (o `edge://extensions` en Edge).
2. Activa el interruptor **Modo de desarrollador**, arriba a la derecha.
3. Pulsa **Cargar descomprimida** y selecciona la carpeta `ecom_adblocker`.
4. La extensión aparece en la lista y se activa sola.



Qué hace al instalarse: guarda sus ajustes por defecto (todo activado, sin listas blancas) y activa las seis listas de bloqueo de red. No crea ficheros fuera de la extensión, no pide ninguna cuenta y no se conecta a ningún servidor.

Al desinstalar se borra todo: los ajustes, las listas blancas y el contador. No deja rastro.

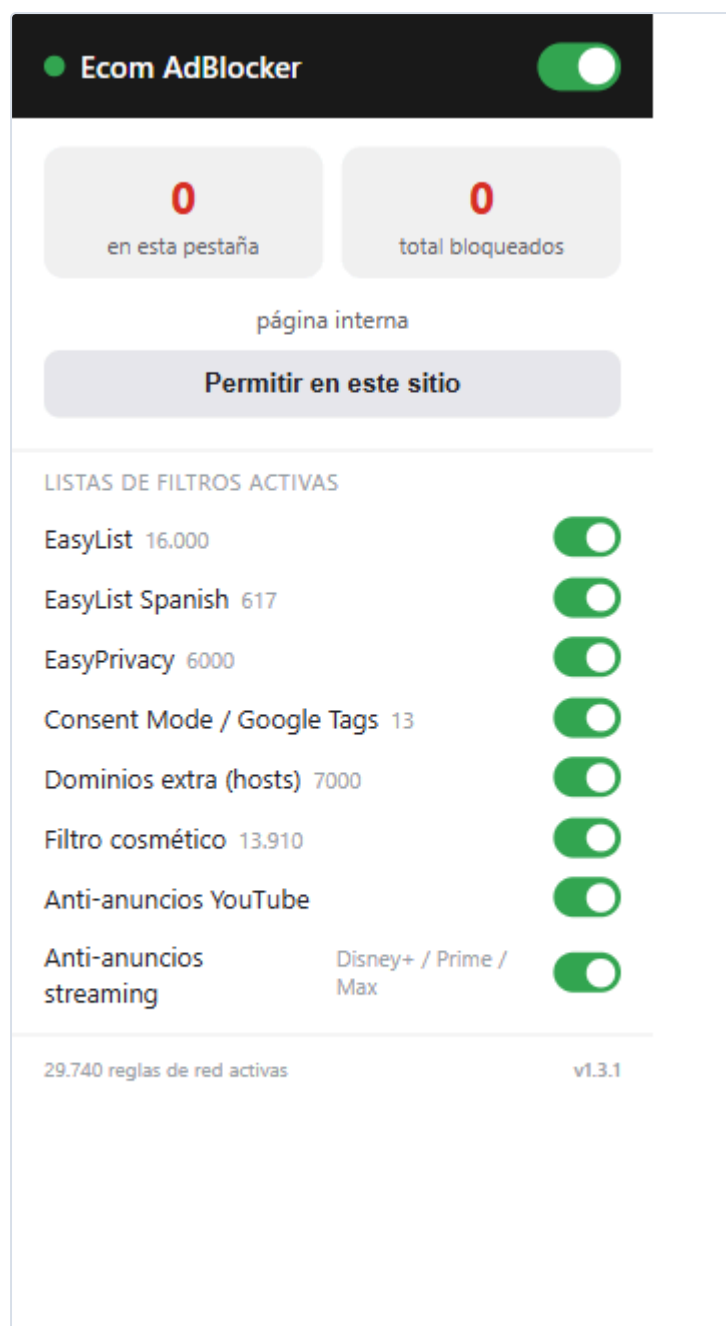
Permisos que pide, y por qué

Permiso	Para qué sirve
<code>declarativeNetRequest</code>	Aplicar las listas de bloqueo
<code>declarativeNetRequestFeedback</code>	Contar cuántas peticiones se han bloqueado en cada pestaña
<code>storage</code>	Guardar tus ajustes y tus listas blancas
<code>activeTab</code>	Saber el dominio de la pestaña actual, solo cuando abres la ventana emergente

No pide acceso permanente a todas las webs, ni a las cookies, ni al historial, ni a las pestañas. **No hace ninguna petición a Internet:** todas las listas de filtros viajan dentro de la extensión.

4. Configuración

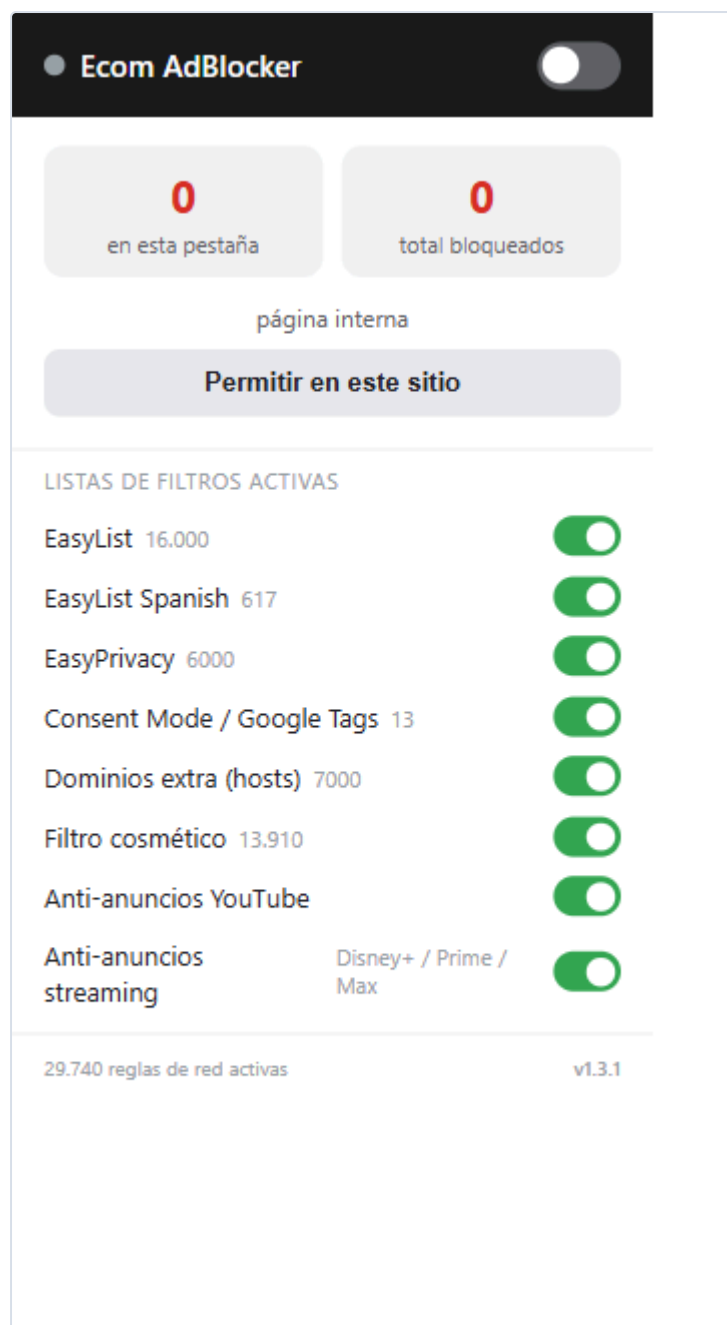
Toda la configuración está en la ventana emergente, la que se abre al pulsar el icono de la extensión en la barra del navegador. No hay pantalla de opciones aparte.



El interruptor de arriba

Enciende y apaga la extensión entera. Al apagarlo se desactivan las seis listas de red de golpe, deja de aplicarse el filtro cosmético y se paran los guiones de streaming; el punto verde de la izquierda se apaga y el

icono de la barra se pone gris con la etiqueta **off**. Es el botón para cuando una web se rompe y quieres descartar rápidamente que sea culpa del bloqueador.



Los dos contadores

- **En esta pestaña:** peticiones bloqueadas desde que la página empezó a cargar. Se pone a cero en cada recarga. El mismo número aparece en el icono de la barra.
- **Total bloqueados:** acumulado desde que instalaste la extensión.

El contador de la pestaña se refresca al terminar de cargar la página, al cambiar de pestaña y al abrir la ventana emergente. **No sube en tiempo real** mientras miras el número: si quieres verlo actualizado, cierra y vuelve a abrir la ventana.

Permitir en este sitio

Añade el dominio de la pestaña actual a la lista blanca y **recarga la página**. En un sitio de la lista blanca no se bloquea nada: ni red, ni filtro cosmético, ni los guiones de streaming. El botón cambia a «Sitio permitido (quitar)» para poder deshacerlo.

Es lo que hay que usar cuando una tienda, un banco o un periódico deja de funcionar bien. Antes de dar por hecho que la culpa es del bloqueador, pruébalo: si con el sitio permitido funciona, ya sabes de qué se trataba.

Pagina de prueba del bloqueador

Contenido normal: esto TIENE que verse.

Segundo bloque de contenido normal.

Anuncio adsbygoogle

Bloque .advertisement

Slot GPT (div-gpt-ad)

Bloque .publicidad

Bloque #ad-lateral

Ojo con los `iframe`. La lista blanca se aplica según el dominio del marco que hace la petición. Si una web permitida incrusta contenido de un tercero dentro de un `iframe`, ese `iframe` sigue filtrándose. En la práctica casi nunca importa, pero explica los casos raros en los que un sitio permitido sigue viéndose recortado.

Permitir anuncios en este canal

Este botón **solo aparece en YouTube**, y solo cuando la extensión ha podido identificar el canal del vídeo que estás viendo. Al pulsarlo, ese canal concreto pasa a ver sus anuncios con normalidad: es la forma de apoyar económicamente a un creador sin renunciar a bloquear el resto de YouTube.

Los canales permitidos se listan debajo, cada uno con una **X** para quitarlo. La sección está oculta mientras la lista esté vacía.

Las ocho listas

Cada interruptor se puede apagar por separado. El número gris que va al lado es cuántas reglas o selectores aporta esa lista.

Interruptor	Reglas	Qué apaga y cuándo conviene
EasyList	16.000	La lista principal de publicidad. Es la que más quita; apagarla deja pasar casi todo.
EasyList Spanish	617	Publicidad de webs en español. Incluye 57 excepciones que evitan romper sitios concretos.
EasyPrivacy	6.000	Rastreadores y analítica. Apágala si sospechas que está estorbando a alguna herramienta de medición que necesites.
Consent Mode / Google Tags	13	Google Tag Manager, Google Analytics 4 y el Consent Mode. La primera que hay que apagar si una web no carga bien: hay sitios que cargan funcionalidad real (no solo analítica) a través de Tag Manager.
Dominios extra (hosts)	7.000	Lista de dominios publicitarios sueltos que no cubren las anteriores.
Filtro cosmético	13.910	La ocultación de huecos y banners y la limpieza de fondos. Apágalo si una página aparece descuadrada o con trozos que faltan.
Anti-anuncios YouTube	—	El guion de YouTube.
Anti-anuncios streaming	—	Los guiones de Disney+, Prime Video y HBO Max, los tres a la vez.

Los cinco primeros son de bloqueo de red y suman **29.740 reglas** activas, que es lo que indica el pie de la ventana. Los tres últimos no son reglas de red, sino código que se ejecuta dentro de la página.

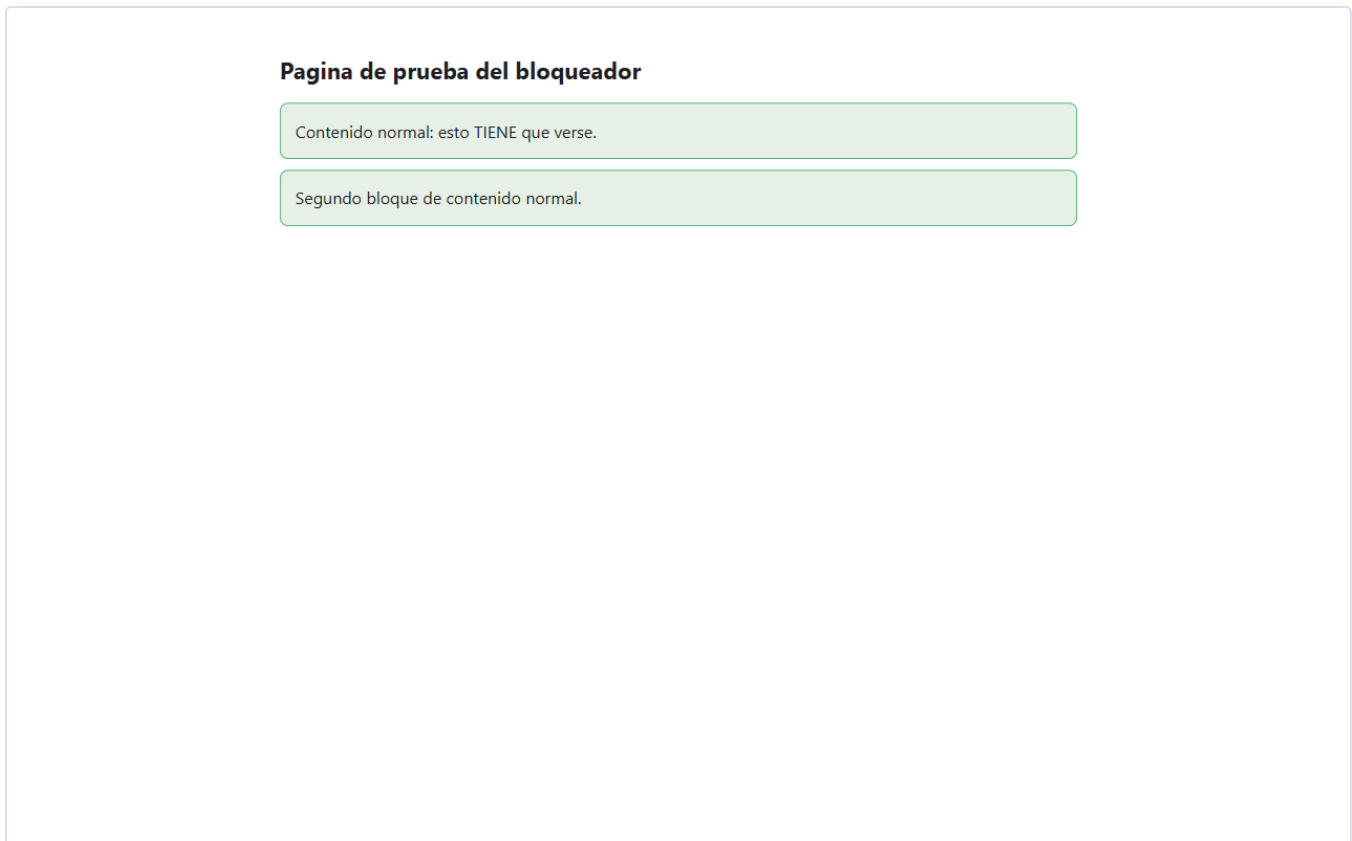
Al apagar una lista, la extensión la desactiva en el acto: no hace falta recargar ni reiniciar el navegador.

5. Cómo se usa, día a día

Lo normal es no tocar nada. Recién instalada bloquea con todo activado y no requiere mantenimiento.

Navegación corriente

Abres una web y los anuncios no están. El número del icono de la barra te dice cuántas peticiones se han cortado en esa página.



En la imagen, una página de prueba con dos bloques de contenido normal (verdes) y cinco bloques de publicidad. Los cinco anuncios han desaparecido y el contenido está intacto: eso es exactamente lo que tiene que pasar.

Cuando una web se rompe

Ve por este orden, parando en cuanto funcione:

1. **Apaga «Consent Mode / Google Tags»** y recarga. Es la causa más frecuente, porque algunas webs cargan funcionalidad de verdad a través de Google Tag Manager.
2. **Apaga «Filtro cosmético»** y recarga, si lo que pasa es que la página se ve descuadrada o falta algún trozo.
3. **Pulsa «Permitir en este sitio»**. Desactiva todo en ese dominio, y la página se recarga sola.
4. **Apaga el interruptor general**, para confirmar que el problema era del bloqueador y no de la propia web.

Si has llegado al paso 3, deja el sitio permitido y vuelve a encender lo que apagaste en los pasos 1 y 2: así el resto de la navegación sigue protegida.

Vídeo

En YouTube, Disney+, Prime Video y HBO Max no hay que hacer nada: los anuncios se saltan solos. Cada plataforma se trata de forma distinta porque cada una monta los anuncios de otra manera:

- **YouTube:** pulsa el botón «Saltar» si aparece, adelanta el anuncio hasta el final y, si aún no puede, lo pone a velocidad $\times 16$ y lo silencia.

- **Prime Video y Disney+:** leen cuánto dura el anuncio y adelantan el vídeo exactamente eso. Como en esas plataformas el anuncio ocupa su propio tramo de la línea de tiempo, no se pierde nada de la película. En Prime el salto está limitado a 90 segundos por tanda, porque saltos mayores hacen fallar al reproductor de Amazon.
- **HBO Max:** aquí **no se adelanta**, a propósito. En HBO el anuncio va incrustado en el mismo flujo que el contenido, y adelantar se comería tantos minutos de película como dure el anuncio. En su lugar se silencia, se acelera y se tapa con una cortina negra que pone «Publicidad bloqueada» y la cuenta atrás. Al acabar el anuncio se restauran el volumen y la velocidad que tenías.

Apoyar a un creador de YouTube

Abre uno de sus vídeos, pulsa el icono de la extensión y **Permitir anuncios en este canal**. La página se recarga y a partir de ahí ese canal cobra por sus anuncios. Se puede quitar cuando quieras desde la lista de canales de la propia ventana.

6. Qué ve el usuario en la web

Nada. La extensión no añade botones, ni marcas de agua, ni avisos en las páginas que visitas. Lo único que dibuja es la cortina negra de HBO Max mientras dura el anuncio, y solo ahí.

Lo que sí se nota es la ausencia: donde había un banner no queda ni el hueco, porque el filtro cosmético colapsa el contenedor además de ocultarlo.

7. Preguntas frecuentes y problemas

El contador está a cero y sé que ha bloqueado cosas. Se refresca al terminar de cargar la página, al cambiar de pestaña y al abrir la ventana emergente, no en tiempo real. Cierra y vuelve a abrir la ventana. Si sigue a cero, mira que el interruptor general esté encendido.

Una tienda no me deja pagar. Pulsa «Permitir en este sitio» y vuelve a intentarlo. Ninguna de las listas incluidas bloquea pasarelas de pago (se ha comprobado expresamente: PayPal, Stripe, Redsys, Adyen, Klarna, Bizum, reCAPTCHA y las CDN habituales no aparecen en ninguna regla de bloqueo), pero una web puede cargar su proceso de pago a través de Google Tag Manager, que sí se bloquea.

Una página se ve descuadrada. Apaga «Filtro cosmético» y recarga. Ese es el que aplica las reglas de estilo.

Mi web ya no aparece en Google Analytics. Es lo esperado: la lista «Consent Mode / Google Tags» bloquea Google Tag Manager y Analytics. Apágala si necesitas medir tu propia web mientras navegas.

¿Envía la extensión algo a algún servidor? No. No hay una sola llamada a Internet en todo el código; las listas van dentro del paquete. Se ha verificado buscando `fetch`, `XMLHttpRequest`, `sendBeacon` y direcciones `http://` o `https://`: la única aparición es la carga de su propio fichero de filtros desde dentro de la extensión.

¿Puede una web saber que llevo este bloqueador? Por el fichero de filtros, ya no: desde la versión 1.3.1 ese recurso usa una dirección que rota en cada sesión y las páginas no pueden pedirlo. Como cualquier bloqueador, la web puede seguir deduciéndolo si comprueba que un anuncio no le ha cargado; eso es inevitable.

¿Por qué YouTube a veces enseña medio segundo de anuncio? El guion tiene que esperar a que el reproductor marque que hay un anuncio, y comprueba cada 250 milisegundos. Ese margen es inevitable.

Se me ha llenado la lista de canales de YouTube. Cada canal tiene su X al lado en la propia ventana emergente.

¿Se actualizan solas las listas? No. Van dentro del paquete y se refrescan al entregar una versión nueva de la extensión.

8. Seguridad

Se ha revisado el `manifest.json`, el service worker, los cinco guiones de contenido, la ventana emergente y las seis listas de reglas de red (29.740 reglas). Lo corregido y lo comprobado:

Lo que se ha arreglado

1. Inyección de CSS en las páginas visitadas, y filtro cosmético caído en 55 dominios

`content/cosmetic.js`, función `applyExtFilters()`. El código construía la hoja de estilos concatenando en una sola cadena todos los selectores de la lista y añadiendo `{display:none!important;}` al final. En `cosmetic_filters.json` hay **84 entradas que no son selectores sino reglas de CSS completas**, con llaves incluidas (`body {padding-top:0;}`, `.nav.sticky {top:0px;}` ...). Al concatenarlas, esas llaves **cierran la regla antes de tiempo** y aplican declaraciones arbitrarias a la página. Se reprodujo en el navegador: con la lista `['.ad-uno', 'body {padding-top:0;}', '.ad-dos']` el resultado era la regla `.ad-uno, body { ... }` — se pierde `.ad-dos` y se acaba aplicando el estilo a `body`. Y como en CSS **un solo selector inválido anula la regla entera**, en los **55 dominios** que tienen una de esas 84 entradas se caían también los **13.910 selectores genéricos**: en esos sitios el filtro cosmético no hacía absolutamente nada. *Arreglo*: se descartan los selectores que llevan llaves, comentarios de CSS o pseudoclases de Adblock Plus (`:has-text`, `:-abp-`, `:xpath`, `:style( ...)`, y las reglas se insertan con `CSSStyleSheet.insertRule()` por lotes de 100, reintentando uno a uno si un lote falla. Así ningún selector defectuoso puede tumbar a los demás ni inyectar estilos. Verificado: **13.910 selectores activos** repartidos en 140 reglas.

2. Cualquier web podía detectar la extensión `manifest.json`, `web_accessible_resources`.

`content/cosmetic_filters.json` estaba expuesto a `<all_urls>` con dirección fija, así que cualquier página podía pedirlo y saber con certeza que tenías este bloqueador instalado, para servirte contenido anti-bloqueo o para perfilarte. *Arreglo*: `"use_dynamic_url": true`, que hace rotar la dirección en cada sesión. Verificado en el navegador: una página web recibe ahora `Failed to fetch` al intentarlo, mientras la extensión sigue cargando sus filtros con normalidad.

3. Permiso `<all_urls>` que no hacía falta `manifest.json`. La extensión pedía `host_permissions: ["<all_urls>"]`, es decir «leer y cambiar todos tus datos en todos los sitios web». Ese permiso solo se usaba para una cosa: leer el dominio de la pestaña actual al abrir la ventana emergente. *Arreglo*: sustituido por

`activeTab`, que concede ese acceso **solo a la pestaña activa y solo cuando el usuario pulsa el icono**. Como red de seguridad, si Chrome no rellenara la dirección, la ventana se la pide al guion de contenido mediante un mensaje `getHost` (añadido en `cosmetic.js`, solo responde el marco principal). Verificado: el respaldo funciona y devuelve el dominio correcto.

4. Una entrada mal escrita en la lista blanca la anulaba entera `background/service_worker.js`, función `applyWhitelist()`. Los dominios se pasaban tal cual a `initiatorDomains`, que solo admite dominios en minúsculas, sin puerto, sin ruta y sin punto inicial. Un único valor inválido guardado en los ajustes hacía que `updateDynamicRules()` lanzara una excepción, y la lista blanca **completa dejaba de aplicarse en silencio**: el usuario vería sus sitios permitidos en la lista y seguirían bloqueados. *Arreglo*: función `cleanDomains()` que normaliza (minúsculas, quita puerto y ruta), comprueba contra una expresión regular y descarta duplicados. Verificado metiendo basura a propósito (`'HTTP://Ma10:8080/x'`, `''`, `'..'`, `'Ejemplo.COM'`): la lista sobrevive con los dominios buenos correctamente saneados.

5. Mensajes internos sin comprobar el remitente `background/service_worker.js` y `content/youtube.js`. Los receptores de `chrome.runtime.onMessage` atendían cualquier mensaje sin mirar quién lo enviaba, y el service worker usaba el `tabId` recibido sin validarlo. *Arreglo*: se comprueba `sender.id === chrome.runtime.id` en los tres receptores y se comprueba que el `tabId` sea un entero no negativo.

6. Ficheros que no debían viajar en el paquete Dentro de la carpeta de la extensión había **4 MB** de material ajeno: `manifest.zip` (una compilación vieja de la v1.1.0, 612 KB), `tools/convert.js` (el conversor de listas), `store/` (material para la ficha de la tienda) y `_metadata/` (3,3 MB de índices que genera Chrome solo). Todo eso se habría publicado con la extensión. *Arreglo*: movidos a `modulos` `IA/utiles/ecom_adblocker/`; `_metadata/` eliminado, ya que Chrome lo regenera.

7. Versión y documentación desincronizadas La ventana emergente mostraba «v1.1.0» escrita a mano con el `manifest.json` en 1.3.0, y el `README.md` documentaba tres listas de seis y daba cifras de reglas que no eran las reales. *Arreglo*: la versión se lee ahora de `chrome.runtime.getManifest()`, y el `README.md` se ha rehecho con los datos comprobados.

8. Interruptor de streaming que no existía Los guiones de Disney+, Prime Video y HBO Max consultaban `lists.streaming`, pero esa clave no estaba en los valores por defecto ni tenía ningún interruptor: **no había forma de desactivarlos**. *Arreglo*: añadida a los valores por defecto y añadido su interruptor en la ventana emergente.

Lo que se ha revisado y estaba bien

- **Sin código dinámico**: no hay `eval()`, `new Function()`, `innerHTML` con datos, ni `document.write()`. El único `innerHTML` es un `= ''` para vaciar un contenedor.
- **Sin telemetría ni servidores propios**: cero llamadas a Internet. No hay claves de API ni credenciales en el código.
- **Sin permisos peligrosos**: no usa `cookies`, `webRequest`, `history`, `bookmarks`, `downloads`, `management`, `debugger` ni `scripting`.
- **Sin puerta de entrada externa**: no hay `onMessageExternal` ni `externally_connectable`, así que ninguna web puede hablar con la extensión.
- **La ventana emergente escapa correctamente**: los nombres de canal y los dominios se pintan con `textContent`, nunca con `innerHTML`, de modo que un nombre de canal con HTML dentro no puede

ejecutar nada.

- **Las reglas de red no rompen tiendas:** se cruzaron las 29.740 reglas contra una lista de dominios sensibles (PayPal, Stripe, Redsys, Adyen, Braintree, Klarna, Bizum, reCAPTCHA, hCaptcha, cdnjs, jsDelivr, Google Fonts, bancos españoles...). Las coincidencias encontradas son todas legítimas: complementos de jQuery para servir anuncios (`jquery.dfp.js` , `jquery.popunder.js`), píxeles de analítica de Shopify y Magento y dominios de anuncios detrás de Cloudflare. **Ninguna regla bloquea una pasarela de pago, un captcha ni una CDN de librerías.**
- **El contador y los permisos que implica:** `declarativeNetRequestFeedback` permite a la extensión consultar qué peticiones ha bloqueado. Es lo que hace funcionar el contador, y provoca que Chrome muestre un aviso de permisos al instalar. Los datos no salen del navegador. Se deja tal cual porque es la única forma de que el contador funcione en una extensión publicada.

Lo que queda pendiente

- **Margen de reglas casi agotado.** Las 29.740 reglas estáticas están a 260 del tope de 30.000 que garantiza Chrome. Añadir reglas a cualquier lista obligará a quitarlas de otra, o a repartir las listas y rotarlas. No es un fallo, pero condiciona cualquier ampliación futura.
 - **La lista blanca no cubre los `iframe` de terceros.** Se aplica según el dominio del marco que hace la petición, así que un `iframe` ajeno dentro de un sitio permitido sigue filtrándose. Corregirlo obliga a cambiar cómo cada marco decide si actúa, y eso cambia el comportamiento del producto: se deja documentado y sin tocar.
 - **Las 84 entradas defectuosas siguen en `cosmetic_filters.json`.** El código ya las descarta sin consecuencias, pero el fallo real está en el conversor (`utils/ecom_adblocker/tools/convert.js`), que traduce mal los filtros `:style()` de EasyList. Conviene arreglarlo antes de regenerar las listas.
 - **Los guiones de streaming dependen de los selectores de cada plataforma.** YouTube, Disney+, Prime Video y HBO Max cambian su código con frecuencia; cuando lo hagan, habrá que revisar los selectores. Es inherente a esta clase de extensión.
-

9. Ficha técnica

Ficheros

```
ecom_adblocker/
  manifest.json           Manifest V3
  background/service_worker.js listas, lista blanca y contador
  content/cosmetic.js     ocultación, limpieza de fondos y respuesta getHost
  content/cosmetic_filters.json 13.910 selectores genéricos + 9.165 dominios
  content/youtube.js      YouTube + lista blanca de canales
  content/primevideo.js   Amazon Prime Video
  content/disneyplus.js   Disney+
  content/hbomax.js       HBO Max / Max
  rules/network_rules.json 110 reglas curadas
  rules/easylist.json     16.000 reglas
  rules/easylist_es.json  617 reglas (560 bloqueo + 57 excepciones)
  rules/easyprivacy.json  6.000 reglas
  rules/google_tags.json  13 reglas
  rules/extra_domains.json 7.000 reglas
  popup/popup.html · popup.css · popup.js
  icons/icon16.png · icon48.png · icon128.png
  README.md
```

Ajustes guardados (`chrome.storage.local`)

Clave	Tipo	Por defecto	Qué es
<code>enabled</code>	booleano	<code>true</code>	Interruptor general
<code>whitelist</code>	lista	<code>[]</code>	Dominios permitidos
<code>ytWhitelist</code>	lista	<code>[]</code>	Canales de YouTube permitidos
<code>lists</code>	objeto	todo <code>true</code>	<code>easylist</code> , <code>easylist_es</code> , <code>easyprivacy</code> , <code>google_tags</code> , <code>extra_domains</code> , <code>cosmetic</code> , <code>youtube</code> , <code>streaming</code>
<code>totalBlocked</code>	número	<code>0</code>	Acumulado de bloqueos

Guiones de contenido y dónde entran

Guion	Dominios	Momento	Marcos
cosmetic.js	<all_urls>	document_start	todos
youtube.js	*.youtube.com	document_start	solo el principal
hbomax.js	*.hbomax.com , *.max.com	document_start	solo el principal
primevideo.js	*.primevideo.com , *.amazon.com , *.amazon.es	document_start	todos
disneyplus.js	*.disneyplus.com	document_start	todos

Reglas de red

- Seis conjuntos estáticos, **29.740 reglas** en total, todas de acción `block` con prioridad 1, salvo 57 excepciones (`allow` , prioridad 2) en EasyList Spanish.
- La lista blanca se implementa como **una regla dinámica** (id `100001` , acción `allow` , **prioridad 3**) con todos los dominios permitidos en `initiatorDomains` . La prioridad superior es lo que la hace ganar a las reglas de bloqueo.
- Los conjuntos se activan y desactivan uno a uno si falla la operación en lote, para que un identificador inválido no deje todas las listas apagadas.

Mensajes internos

Mensaje	De → a	Devuelve
getTabCount	ventana emergente → service worker	{count}
ytGetChannel	ventana emergente → youtube.js	{channel, name}
getHost	ventana emergente → cosmetic.js	{host}

Los tres receptores comprueban `sender.id === chrome.runtime.id` .

Sin registros ni cron

La extensión no escribe ficheros de registro ni tiene tareas programadas. Los únicos mensajes que deja son avisos de consola (`console.warn` y `console.debug`) para diagnóstico, visibles en las herramientas de desarrollo del service worker.